# Systems programming lesson 2

## COM 312: BSc Computer science/BSc IT

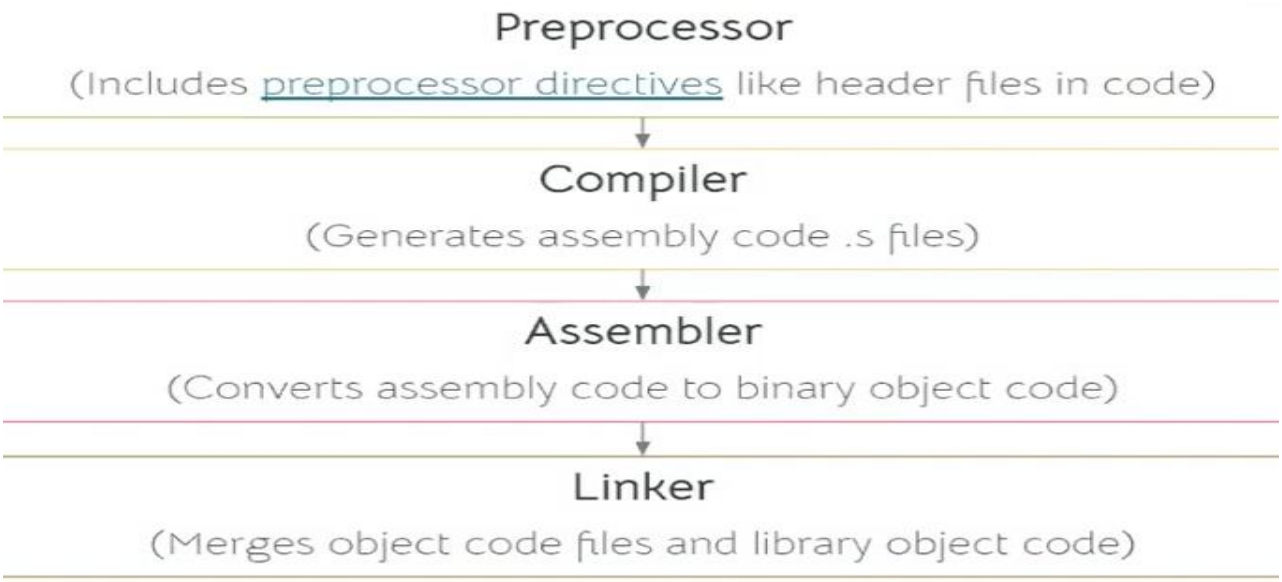
**Topics area coverage.**

- **Preparing your tool Box**
- **GNU gcc and gdb**
- **NASM (Netwide Assembler)**
- **Unix readlf, objdump, od, nm, idd and id config**
- **Creating your own static and dynamic libraries**

## C code compilation process using GNU Gcc (compiler collection)

**How is C code compiled?**

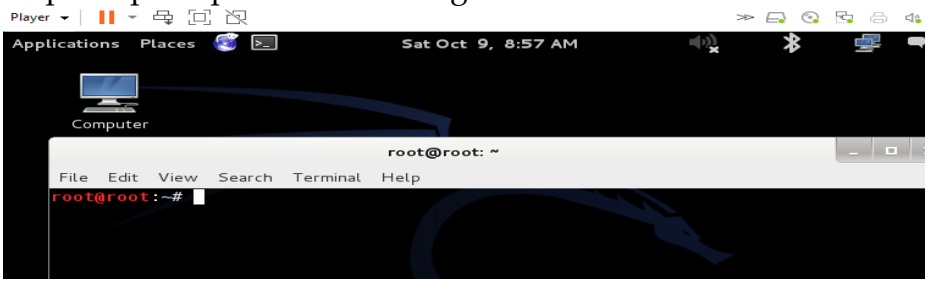
**Four steps process.**




**Open your kali linux Tool**

**Click >terminal**

Step 1. Open up a terminal using kali linux



**Step 2. Use a text editor to create the C source code. (using nano editor)**

Type the command

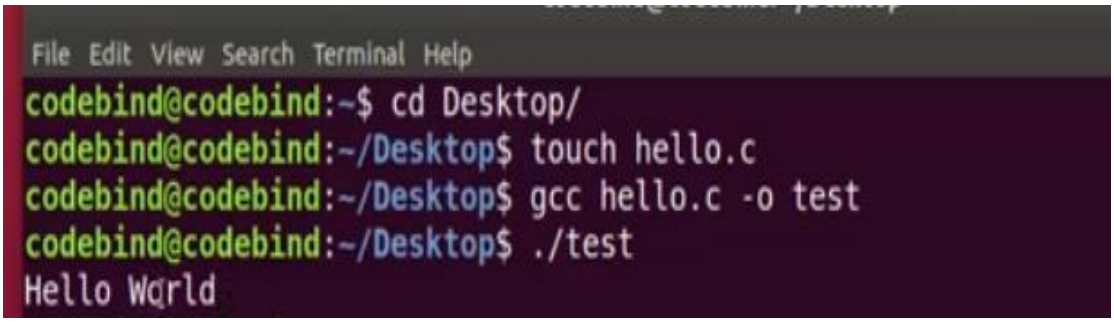
nano hello.c

and enter the C source code below:

```
#include<stdio.h>
Main ()
{ printf ("Hello World\n");
Return 0;
}
```

Close the editor window.

**Or**




**Linux commands**

**cd**- change directory

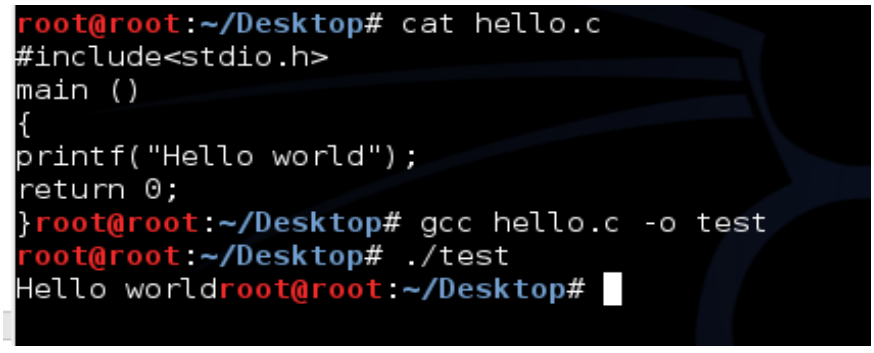
**touch** – the touch command is often used just to create an empty file.

**Gcc**- Gnu compiler collection.

**-o** – flag for output after compilation

**./**- forward slash command for running the test binary file.

**Cat**- concatenate file and print on the standard output.




Step 1 and 2 : Preprocessor to compiler(to generate assembly code.s files)

```
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ vi myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ gcc -S myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c  myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls -l *.s
-rw-r--r-- 1 alohachurchill alohachurchill 657 Sep 16 23:57 myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ cat myProgram.s | less
```

Unix commands

**Ls**- list directory contents (list information about FILES current directory)

**Vi myprogram.c** – vim editor enables viewing of program code

**Gcc-s myprogram.c** – to make assembler code(low level programming) from our C File.

**Ls**- converting myprogram.c to my program.s File

**Ls –l *.s** – searches for all assembly Files

**Cat myprogram.s |less-** opposite of more (to view assembly hello world code)

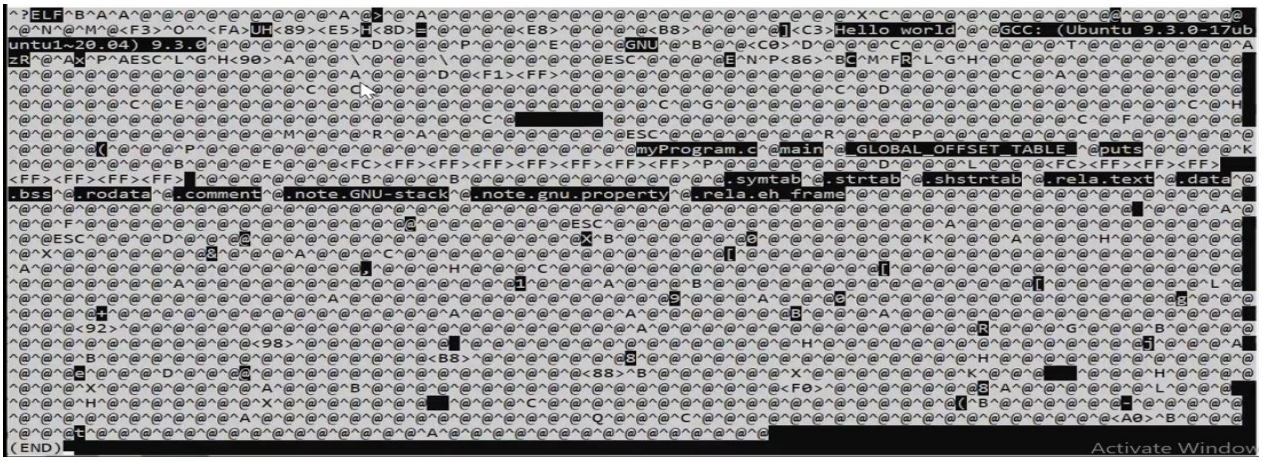
```
.LC0:
        .string "Hello world"
        .text
        .globl  main
        .type   main, @function
main:
.LFB0:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        leaq    .LC0(%rip), %rdi
        call    puts@PLT
        movl    $0, %eax
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE0:
        .size   main, .-main
        .ident  "GCC: (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0"
        .section        .note.GNU-stack,"",@progbits
        .section        .note.gnu.property,"a"
        .align 8
        .long    1f - 0f
        .long    4f - 1f
:_
```

**Step 3 – Assembler**

Converts assembly code to machine object code

```
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ vi myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ gcc -S myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c  myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls -l *.s
-rw-r--r-- 1 alohachurchill alohachurchill 657 Sep 16 23:57 myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ cat myProgram.s | less
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ gcc -c myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c  myProgram.o  myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ cat myProgram.o | less
```

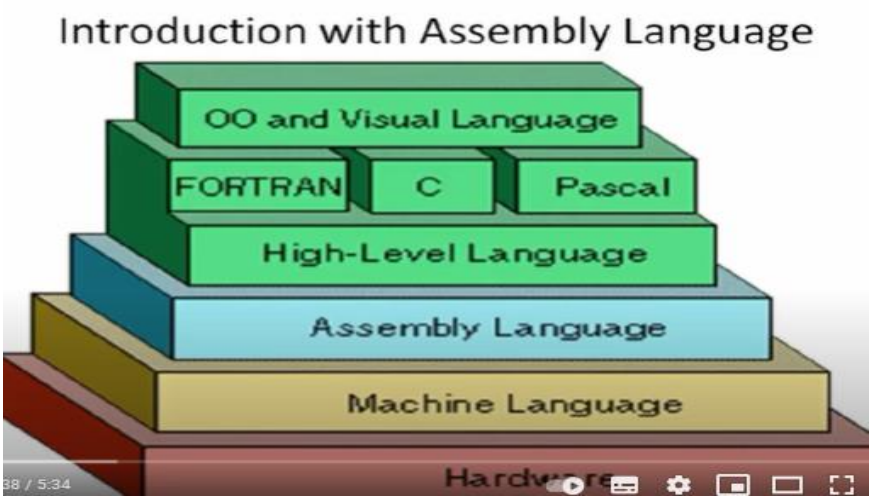
**Object code output**




**Last step 4: linker** – merges object code files and libraly object code

```
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ vi myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ gcc -S myProgram.c
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c  myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls -l *.s
-rw-r--r-- 1 alohachurchill alohachurchill 657 Sep 16 23:57 myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ cat myProgram.s | less
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ gcc -c myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram.c  myProgram.o  myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ cat myProgram.o | less
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ gcc myProgram.c -o myProgram
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ls
myProgram  myProgram.c  myProgram.o  myProgram.s
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ ./myProgram
Hello world
alohachurchill@DESKTOP-7RB6R7D:~/CSProjects/SecondYear/CS2400CA$ cat myProgram | less
```

## NASM (netwide Assembler)




- What is assembly language and its importance?
- What you need to know to start writing your first assembly program.

### What is assembly language and its importance?

- Processor –Dependent
- Low level programming language (Human Readable machine language)
- Assembly code is converted into executable machine code by a utility program referred to as an assembler
  - Each family of **CPUs have its own set of instructions** or Assembly language for Handling Various operations such as
    i. Getting input from keyboard
    ii. Performing various task
    iii. Displaying information on the screen

There are main different types of Assembly languages for different types of CPU Architecture









**ARM –** Architecture used on phones among others (i.e ios, Android)

**MIPS**- Architecture used in video games console among others (i.e sony PlayStation)

**X86 –** Architecture used on intel PCs and other (16 bit to 64bit)

**Why Assembly?**

- Direct communication with the hardware
- Writing device driver
- Access to specialized processor instructions
- Operating systems design
- Embedded systems programs
  i. e A good example is Aviation industry uses a lot of assembler like computer Assembler Engineer

### learning Assembly will help us understand

- How data is represented in memory and other external devices
- How programs interface with your operating system, processor, and BIOS
- How the processor accesses and executes instructions?
- How instructions access and process data
- How a program accesses external devices?

### Basic features of a computer Hardware

- Registers
- Memory
- Processor

### Understanding Registers

- One of the main tools to write programs in X86 assembly are the **processor registers.**
- You can, informally, think of **registers** as variables that are built in the processor
- Using **registers** instead of memory to **store values**, makes the process faster and more efficient.

**An assembly program can be divided into three sections −**

- The **data section**, - declaring initialized data or constants. This data does not change at runtime. You can declare various constant values, file names, or buffer size, etc
- **The bss section**, - The **bss** section is used for declaring variables

- The **text** section. - The **text** section is used for keeping the actual code. This section must begin with the declaration **global _start**, which tells the kernel where the program execution begins.

  syntax

```
section.text
   global _start
_start:
```

**The Hello World Program in Assembly**

```
section .text
   global _start     ;must be declared for linker (ld)

_start:              ;tells linker entry point
   mov  edx,len      ;message length
   mov  ecx,msg      ;message to write
   mov  ebx,1        ;file descriptor (stdout)
   mov  eax,4        ;system call number (sys_write)
   int  0x80         ;call kernel

   mov  eax,1        ;system call number (sys_exit)
   int  0x80         ;call kernel

section .data
msg db 'Hello, world!', 0xa  ;string to be printed
len equ $ - msg      ;length of the string
```

Output

```
Hello, world!
```

Code explanation

**Mnemonic in assembly language**

**Mnemonic language –** memory aid **to represent machine operation, or opcode**

**MOV-** move (assign) one value to another

**ADD-** Add two values

**SUB-** subtract one value from another

**MUL-**multiplying two values

**JMP-** jump to a new location

**CALL-**call a procedure

**EAX-** to store the return value of a function and as a special register for certain calculations**.** (**general purpose register**)

**EBX-**base pointer to the data section

**ECX-**counter for string and loop operations

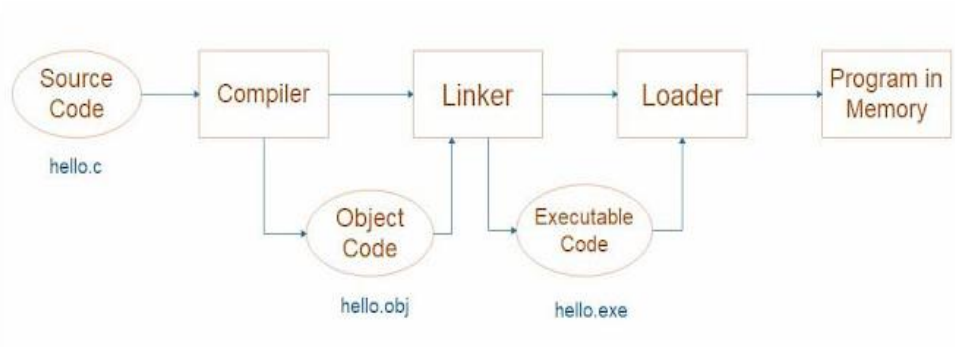
**EDX-** 1/0 pointer

## Unix readlf, objdump, od, nm, idd and id config

**Unix read elf- Executable Linking Format.** This is the file which is loaded by the Kernel to create a process.

**NB:** A relocatable file **holds code and data** suitable for linking with other object files to create an executable

- ELF is the file format used for object files (`.o`'s), binaries, shared libraries and core dumps in Linux.
- ELF is used as standard file format for object files on Linux. Prior to this, the a.out file format was being used as a standard but lately ELF took over the charge as a standard.




- An object file is **a computer file containing object code**, that is, machine code output of an assembler or compiler. The object code is usually **relocatable**, and not usually directly executable.
- Since then, the ELF format is in use by several different operating systems. Among others, this includes Linux, Solaris/Illumos, Free-, Net- and OpenBSD,

QNX, BeOS/Haiku, and Fuchsia OS [2]. Furthermore, you will find it on mobile devices running Android, Maemo or Meego OS/Sailfish OS as well as on game consoles like the PlayStation Portable, Dreamcast, and Wii.

ELF supports:

- Different processors (cross plat-form)
- Different data encoding
- Different classes of machines

**What is executable?**

Binary file with instruction to be performed by the CPU

**Why use ELF Files?**

- Generic understanding of how an operating system works
- Development of software
- Digital Forensics and Incident Response (DFIR)
- Malware research (binary analysis)

## Revision Questions

1. Why does Assembly It require less memory and execution time?
2. Why does Assembly language allow hardware-specific complex jobs in an easier way?